arXiv:2512.10501v2 [cs.AI] 12 Dec 2025

ZERO-SHOT 3D MAP GENERATION WITH LLM AGENTS: A DUAL-AGENT ARCHITECTURE FOR PROCEDURAL CONTENT GENERATION

Lim Chien Her[1,*], Ming Yan[1], Yunshu Bai[1], Ruihao Li[1], and Hao Zhang[1]

[1]MiAO

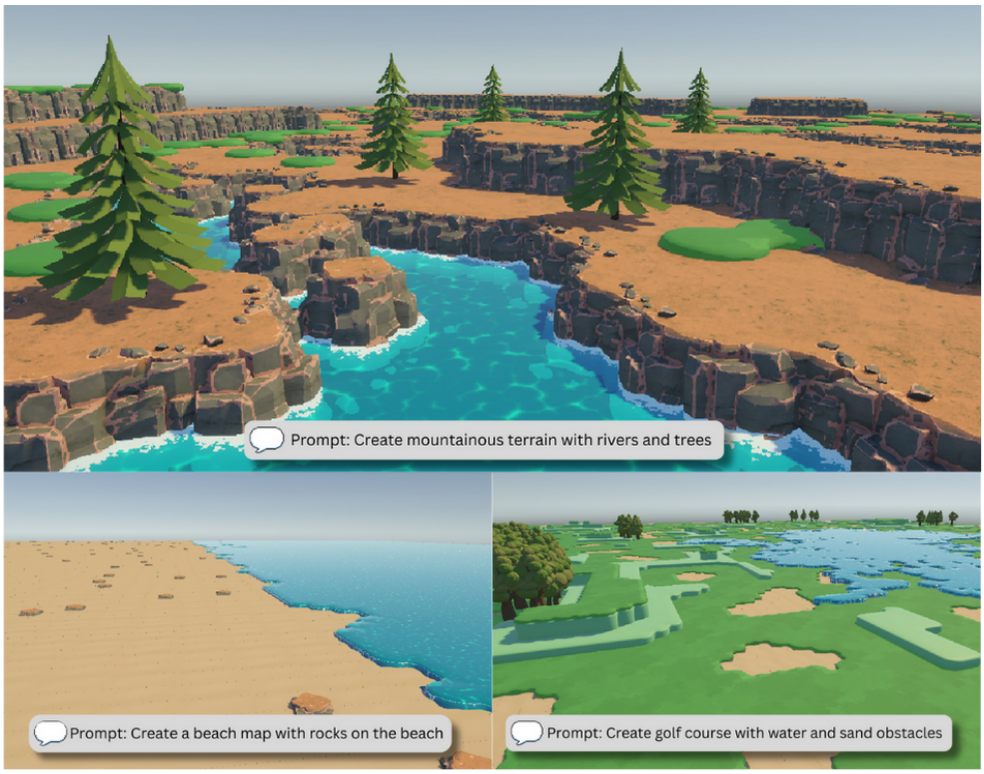


Figure 1: Our proposed training-free architecture generates realistic 3D maps from natural language prompts using procedural content generation in a zero-shot manner.

ABSTRACT

Procedural Content Generation (PCG) offers scalable methods for algorithmically creating complex, customizable worlds. However, controlling these pipelines requires the precise configuration of opaque technical parameters. We propose a training-free architecture that utilizes LLM agents for zero-shot PCG parameter configuration. While Large Language Models (LLMs) promise a natural language interface for PCG tools, off-the-shelf models often fail to bridge the semantic gap between abstract user instructions and strict parameter specifications. Our system pairs an Actor

*Correspondence to: limchienher@miao.company

agent with a Critic agent, enabling an iterative workflow where the system autonomously reasons over tool parameters and refines configurations to progressively align with human design preferences. We validate this approach on the generation of various 3D maps, establishing a new benchmark for instruction-following in PCG. Experiments demonstrate that our approach outperforms single-agent baselines, producing diverse and structurally valid environments from natural language descriptions. These results demonstrate that off-the-shelf LLMs can be effectively repurposed as generalized agents for arbitrary PCG tools. By shifting the burden from model training to architectural reasoning, our method offers a scalable framework for mastering complex software without task-specific fine-tuning.

1 Introduction

Procedural Content Generation (PCG) plays a critical role in modern game development by allowing the scalable creation of sufficient and varied content for contemporary virtual environments, thus significantly reducing manual production requirements [1–3]. PCG operates through generative algorithms that are parameterized and often combined in multi-layered pipelines. The process typically begins with a seed value, ensuring deterministic output, and applies a series of rules to transform basic geometric or semantic primitives into large-scale, internally consistent game environments [4]. By leveraging these predefined rules and algorithms, PCG supports the scalable creation of varied and configurable game environments. Variants of this approach are employed in commercial titles such as *Minecraft* [5], *Diablo* [6] and *Valheim* [7].

Despite widespread adoption [8], authoring effective PCG pipelines remains a challenging task. The artistic and functional qualities of the output are controlled by a dense set of technical parameters that are often opaque, non-intuitive, and lack a direct mapping to high-level design concepts. As such, navigating these high-dimensional parameter spaces [9] and selecting appropriate algorithms [3] require substantial expertise and often involve iterative trial and error [10].

Recent developments in large language models (LLMs) have prompted research into using natural language to control these pipelines [11–13]. However, off-the-shelf LLMs struggle to bridge the semantic gap between abstract user instructions and strict parameter specifications [14]. While adept at linguistic interpretation [15], standard models lack the grounding to assign functionally appropriate values to domain-specific variables, often resorting to speculative guessing based on parameter names. To mitigate this, current approaches rely on extensive fine-tuning [16], reinforcement learning [17], or in-domain pre-training [18]. These methods are resource-intensive [19, 20] and result in rigid models that lack generalizability across different tools or domains [21, 22]. To address these limitations, we introduce a training-free, dual-agent architecture that enables off-the-shelf LLMs to interface with advanced PCG tools in a zero-shot setting, presented in Figure 2. Our system pairs an Actor agent, which translates natural language prompts into initial parameter proposals, with a Critic agent, which evaluates these proposals against algorithmic documentation and constraints. Through a dialogic process, the agents iteratively refine the configuration, allowing the system to resolve ambiguities and correct errors autonomously. This interaction replaces the need for gradient updates with in-context reasoning, ensuring that valid parameters are selected based on logic rather than memorized patterns.

Critically, this approach demonstrates that general-purpose LLMs can be repurposed as flexible, domain-agnostic controllers. By shifting the burden from model training to architectural design, our method requires no task-specific fine-tuning or data collection. This allows for immediate transfer to other new tools simply by swapping the provided reference documentation.

Our primary contributions are summarized as follows:

1. **A Training-Free Framework for Zero-Shot PCG:** We demonstrate that off-the-shelf LLMs can operate complex, domain-specific software solely through the injection of static documentation and reference examples, eliminating the need for resource-intensive fine-tuning or reinforcement learning.
2. **Dual-Agent Actor–Critic Architecture:** We introduce a novel architecture that decouples *semantic planning* (Actor) from *functional verification* (Critic). We show that this dialogic separation of concerns allows the system to autonomously resolve ambiguities and correct parameter hallucinations without human intervention.
3. **Empirical Validation of Reliability:** Through quantitative experiments on 2D and 3D map generation tasks, we establish that our dual-agent approach achieves a 20% higher task success rate compared to single-agent

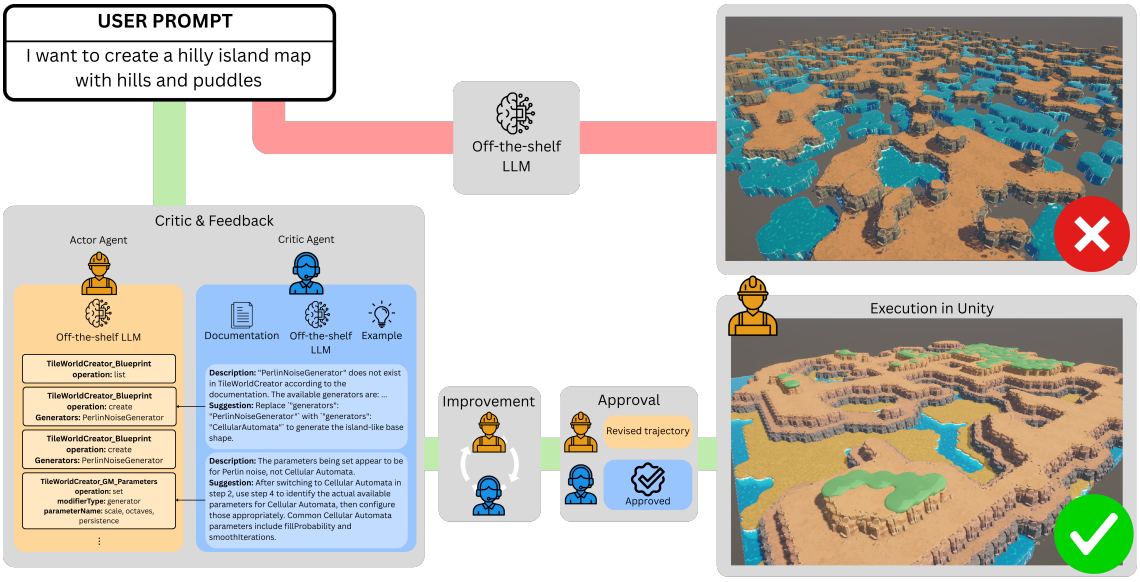


Figure 2: Our proposed architecture employs the use of two agents which interact dialogically to achieve understanding of opaque and non-intuitive parameters of PCG tools. Training-free and in zero-shot, the architecture is able to produce complex 3D maps.

baselines, effectively bridging the semantic gap between natural language and high-dimensional parameter spaces.

2 Related Work

Traditional PCG. Procedural Content Generation (PCG) refers to the algorithmic creation of game content [8]. Since the 1980s, PCG has evolved into three main paradigms: search-based, machine learning (ML)-based, and large language model (LLM)-based approaches [4, 8, 23]. Search-based PCG typically utilizes evolutionary algorithms, where content is evolved via a fitness function that assesses quality against specific criteria [24, 25]. While effective for optimizing well-defined constraints, these methods rely heavily on manually designed fitness functions and lack the semantic flexibility to interpret vague or high-level natural language instructions, limiting their utility in rapid, intent-driven design workflows. Machine Learning (ML)-based approaches employ models such as Generative Adversarial Networks (GANs), Long Short-Term Memory (LSTM) and Autoencoders to learn generative patterns from existing game corpora [23]. While these methods can capture complex stylistic nuances that rule-based systems miss, they suffer from a 'black box' problem: the resulting content is often difficult to control or edit post-generation [26]. Furthermore, training these models requires large annotated datasets of game levels, which are rarely available [26]. This data dependency makes ML-based approaches brittle when applied to new domains without extensive retraining.

LLM-PCG. To incorporate natural language control, prevailing methods rely on fine-tuning or reinforcement learning (RL) to equip models with map generation capabilities. For instance, Muhammad and Togelius [16] finetuned GPT-3 using human-edited data of room levels generated by LLM. Whereas ChatPCG [17] performed Reinforcement Learning with Artificial Intelligence feedback (RLAIF) to align a language model to produce playable maps using PCG. However, these approaches incur significant overhead: fine-tuning requires large quantities of high-quality and domain-specific data [16, 27, 28],which can be costly to produce or curate, while RLAIF depends on a stronger critic model and requires training of a reward model [17, 29]. As a result, these methods are difficult to generalize or repurpose when tools, genres, or design goals change. Off-the-shelf LLM implementations that avoid fine-tuning typically position the model as a natural-language-to-asset bridge rather than as a direct operator of PCG algorithms. Systems like Auxtero [30] and Chen et al. [31] decomposed prompts into object lists for tile-based placement. Crucially, existing research predominantly targets 2D environments, despite modern games prioritizing 3D design [23]. Current 2D work focuses on top-down map or obstacle generation (e.g., Sokoban levels [1, 2, 16]), leaving 3D terrain synthesis underexplored. These pipeline architectures successfully translate user prompts into executable code or asset selection steps, but often sidestep the hardest part of PCG tool use: choosing and configuring generation algorithms via

dense, opaque parameter sets. LLMs are restricted to issuing high-level commands, with the core generative logic and parameterization delegated to hand-engineered components.

In-Context Learning and Tool Use. Unlike traditional ML approaches that require parameter updates, Large Language Models demonstrate the emergent ability to perform tasks via In-Context Learning (ICL), where the model infers rules and patterns directly from the prompt [28]. Recent work has extended this capability to "Tool Use," where LLMs function as controllers for external APIs and software interfaces by interpreting documentation and generating executable syntax zero-shot [32–34]. This paradigm shifts the engineering burden from dataset curation to prompt design. However, most existing research in tool use focuses on basic retrieval or calculator APIs. Our work extends this by investigating whether ICL is sufficient for mastering the high-dimensional, inter-dependent parameter spaces found in professional 3D content creation tools.

Multi-agent Architectures. Recent multi-agent architectures retain this controller-centric paradigm by tethering LLMs to external game engines or modeling software, despite demonstrating improved fidelity in 3D scene generation. For example, UnrealLLM [35] employs agents to translate natural language into Unreal Engine 5's PCG blueprints via a constructed knowledge base, effectively delegating terrain synthesis to the engine. SceneCraft [36] and LL3M [37] generate editable Blender code through planning, retrieval, and coding agents to improve robustness and error handling. 3D-GPT [38] adopts a three-agent design in which one agent identifies key functional elements in map specifications, another resolves ambiguities by elaborating vague descriptions, and a third produces Blender API function calls. While these systems show that multi-agent coordination can enhance reliability and expressivity, they rely on substantial bespoke scaffolding—such as curated knowledge bases, retrieval pipelines, or API-specific adapters. This infrastructure is time-consuming to construct and difficult to transfer to new, high-dimensional parameter spaces common in commercial tools. In contrast, our work enables LLM agents to master such tools through purely in-context learning. By utilizing task demonstrations to guide parameter synthesis without weight updates or bespoke adapters, we democratize LLM-driven PCG for complex 3D terrain generation.

3 Method

We introduce a dual-agent architecture to support zero-shot adaptation of large language models to PCG tools and to improve their ability to interpret and assign values to domain-specific parameters. This framework addresses the challenge of interpreting high-dimensional domain parameters without fine-tuning. The architecture comprises two LLM-based agents, an Actor and a Critic (section 3.1) , functioning within an iterative dialogic loop (section 3.3).

3.1 Dual Agent Architecture

Actor. The Actor functions as the semantic interpreter and generator. Given a user's natural language prompt (P_{user}) describing the desired map, the Actor parses the intent and synthesizes an initial Parameter Trajectory Sequence (S_0). This sequence maps high-level, open-ended design concepts into specific PCG algorithm steps and their corresponding low-level parameter values. By operationalizing abstract descriptors into concrete tool settings, the Actor bridges the semantic gap between human intent and rigid software interfaces.

Critic. The generated trajectory (S_0) is passed to the Critic agent, which acts as a static verifier. The Critic evaluates the proposed parameters against two provided knowledge sources:

1. **API Documentation:** Formal definitions of available PCG algorithms and parameter constraints.
2. **Reference Demonstration:** Validated example of algorithm orchestration for 3D map generation.

Unlike the Actor, which prioritizes semantic alignment, the Critic prioritizes functional correctness. It identifies discrepancies where syntactically valid parameters may be operationally flawed (e.g., out-of-bounds values or incompatible algorithm pairings). As presented in Figure 2, for every detected error, the Critic outputs a structured critique containing a description of the error and a correction suggestion, grounding the generation in authoritative specifications.

3.2 Prompt Formulation

To facilitate zero-shot adaptation, we employ a structured prompt engineering strategy that grounds both agents in domain documentation while enforcing strict output formats.

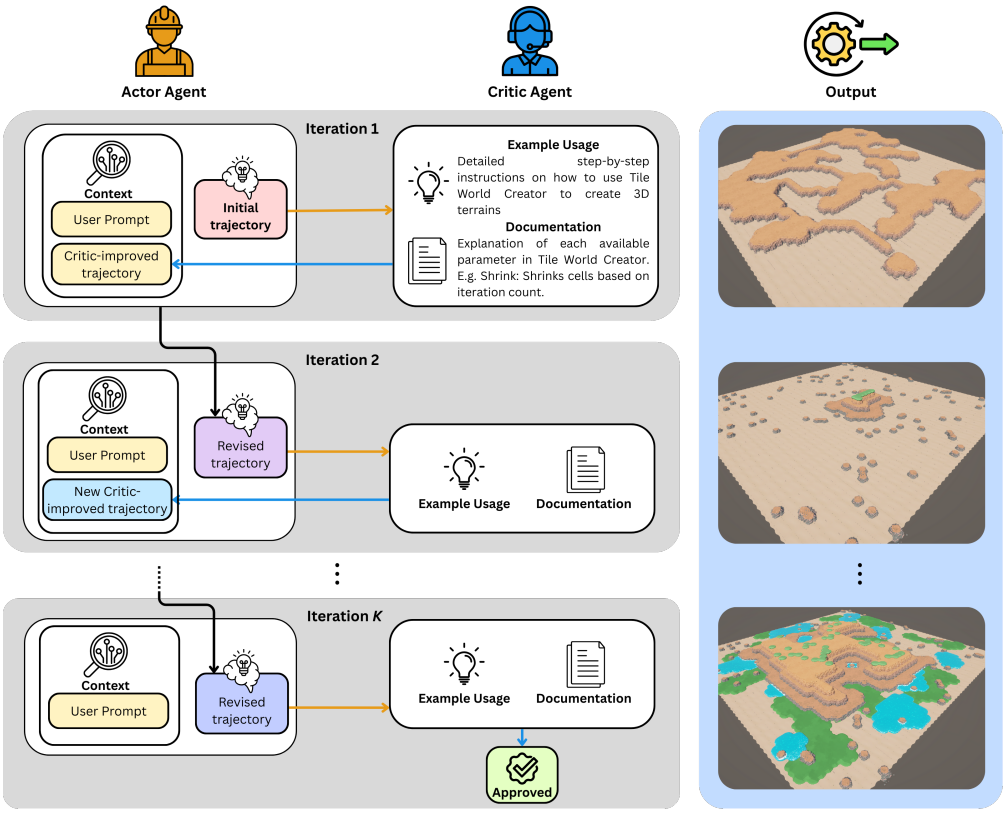


Figure 3: The Actor and Critic interacts dialogically to continuously refine the trajectory until (1) the Critic approves or (2) maximum iterations has been reached. Following which, the Actor creates the map based on the latest trajectory.

Table 1: Operational Comparison of Actor and Critic Agents

Agent	Role	Operational Priority	Rules and Guidelines	Output Structure
Actor	Semantic Interpreter	**Semantic Alignment**: Bridges the gap between abstract user intent and rigid tool settings.	**No Immediate Execution**: Forbidden from executing tools directly. **Risk Assessment**: Must explicitly list assumptions or potential risks.	**Parameter Trajectory** {`trajectory_summary`, `tool_plan`, `risks`}
Critic	Static Verifier	**Functional Correctness**: Prioritizes validity against API documentation over creativity.	**Conservative Certainty**: Only flag "blocking issues" if absolutely certain. **Contextual Review**: Verify against docs and usage examples.	**Structured Critique** {`decision`, `blocking_issues`, `correction_suggestion`}

Actor Prompt Structure. The Actor is initialized with a system instruction establishing its persona as a Unity tool planner. The prompt explicitly decouples planning from execution, strictly forbidding the agent from executing tools itself. Instead, it is tasked with proposing a concrete Execution Trajectory. The Actor is constrained to output a strict JSON structure comprising:

1. **Trajectory Summary:** A high-level overview of the plan.
2. **Tool Plan:** A granular list of sequential steps, where each step defines an `objective`, the specific `tool_name`, the required `arguments`, and an `expected_result` for verification.
3. **Risk Assessment:** A dedicated field for identifying potential blocking risks or missing information before the plan is submitted

Critic Prompt Structure. The Critic functions as a Plugin Expert tasked with validating the Actor's proposed trajectory against provided documentation and usage examples. Its system prompt enforces a comprehensive Review Framework covering five distinct dimensions: Tool Selection, Parameter Correctness, Logic & Sequence, Goal Alignment, and a Certainty Requirement. Crucially, the Critic is instructed to adopt a conservative review policy: it must only flag "blocking issues" if it is absolutely certain of an error, ensuring that the system does not stall due to minor ambiguities or false positives. The Critic's output is a strict JSON object containing a binary `decision` ("approve" or "revise") and a list of `blocking_issues` or `missing_information`, ensuring actionable feedback is passed back to the Actor in the refinement loop.

3.3 Iterative Refinement Protocol

Algorithm 1: Zero-shot Dual-Agent PCG Refinement

Input: User Prompt P_{user}, Documentation D, Usage Examples E, Max Iterations K
Output: Final Parameter Trajectory S_{final}

$Context_{actor} \leftarrow \{P_{user}, D, E\}$
$S_0 \leftarrow \texttt{Actor}(Context_{actor})$ `// Initial generation`
$i \leftarrow 0$
while $i < K$ **do**
  $Feedback \leftarrow \texttt{Critic}(S_i, D, E)$
  if $Feedback = \emptyset$ **then**
    return S_i `// Valid configuration found`
  end
  $Context_{actor} \leftarrow$ UpdateContext$(S_i, Feedback)$
  $S_{i+1} \leftarrow \texttt{Actor}(Context_{actor})$
  $i \leftarrow i + 1$
end
return S_K `// Return best effort if max iterations reached`

The core of our architecture is an internal dialogic feedback loop that allows the system to resolve ambiguities and correct hallucinations progressively. The Actor receives the Critic's feedback combined with the original User Prompt to generate a revised trajectory (S_{i+1}), as shown in Algorithm 1. Through successive iterations, the trajectory becomes increasingly constrained by the functional requirements provided by the Critic, ensuring convergence toward a valid execution plan without requiring gradient updates.

Context Management. To ensure the system remains within the LLM's effective context window across multiple iterations, we implement a *state-replacement strategy*. Rather than appending the entire history of the dialogue, the system maintains a fixed-size context buffer. As shown in Figure 3, the context is updated in-place: the distinct "Initial Trajectory" block is overwritten by the "Revised Trajectory" at the start of each new evaluation cycle. This preserves the most relevant state information—the current best hypothesis and the latest critique—while discarding obsolete trajectories, ensuring consistent inference latency and model performance.

4 Experiments

In this section, we evaluate the performance of our dual-agent Actor–Critic architecture against single-agent baselines. We focus on two key dimensions: (1) **Reliability**, assessing the system's ability to satisfy complex, multi-constraint prompt requirements; and (2) **Efficiency**, analyzing the trade-off between computational overhead and system autonomy. Our results indicate that the Actor–Critic framework not only improves task success rates by 20% but also enhances operational efficiency, reducing the need for human follow-up prompts and decreasing overall token usage by 12.7%.

4.1 Experimental Setup

System Configuration. Our experiments target the *TileWorldCreator* plugin within the Unity Engine. Although our architecture is tool-agnostic, we utilize this plugin to test procedural content generation (PCG) capabilities. Both the Actor and Critic agents are built upon *UGenLah*, a Unity Editor AI assistant that interfaces with the editor via natural language commands (see Figure 4). We employ the *Claude 4.5 Sonnet* model via API for inference. To balance

creativity with instruction adherence, the Actor's temperature is set to 0.4. The Critic's temperature is set to 0.2 to ensure consistent and confident feedback. The maximum number of iteration cycles is set to one for all trials.

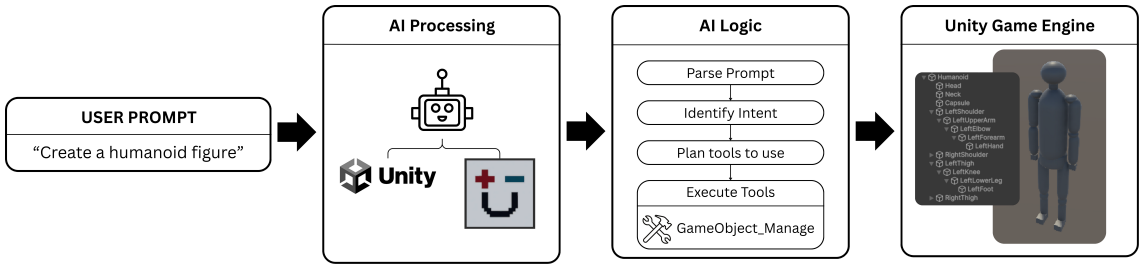


Figure 4: UGenLah system interface. The Unity-based AI assistant has access to more than 30 tools, supporting comprehensive interaction with the Unity Editor, including scene manipulation, asset management, and project configuration.

Experiment I: Complex Task Protocol. We first evaluate the reliability of our proposed Actor–Critic architecture against a single-agent baseline equipped with external resources. Both architectures are tasked with generating a complex 3D mountain map subject to four specific constraints:

- **Terrain:** Formation of a single mountain peak (preventing multiple disconnected peaks).
- **Morphology:** The mountain must consist of exactly three height layers.
- **Detailing:** Grass spots must be applied specifically to the peak layer.
- **Scatter:** Rocks must be scattered in areas not occupied by the mountain.

All requirements are provided in the initial zero-shot prompt. No follow-up prompts are permitted, ensuring that both systems process identical information. We conduct ten independent trials to systematically evaluate success rates and error distribution.

Experiment II: Efficiency Protocol. To isolate the impact of the Critic agent, we compare our dual-agent approach against two single-agent baselines: (1) *Single-Agent (w/ Docs)*, which has access to documentation and example usage, and (2) *Single-Agent (No Docs)*. We evaluate these architectures across four distinct target maps (two 2D and two 3D, see Figure 5), each requiring different PCG algorithms.

Unlike Experiment I, this protocol allows for iterative refinement. If an architecture fails to produce the target map, we provide follow-up prompts describing the observed discrepancies (e.g., "The height layers are disconnected") without offering explicit solutions (e.g., we do not state "reduce the y-offset"). This tests the agent's ability to self-correct based on descriptive feedback.

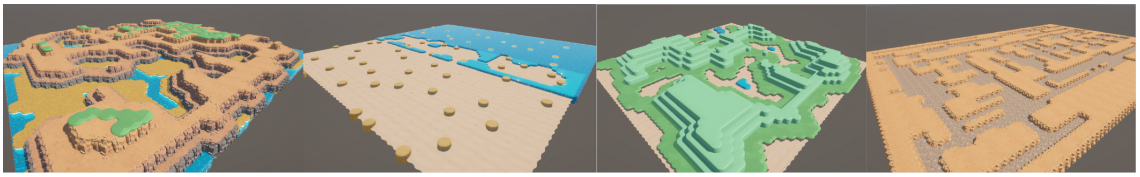


Figure 5: We evaluate the architectures on four distinct map types (two 2D and two 3D). Each map type is generated through a multi-step pipeline that applies a different combination of procedural content generation (PCG) algorithms, thereby assessing the architectures' ability to generalize across diverse generation processes and spatial representations.

Evaluation Metrics. We employ distinct metrics for each experiment to assess both output quality and computational efficiency.

- **Experiment I Metrics:** We report (1) *Task Success Rate* and (2) *Average Mistakes per Run*. A trial is classified as a **Success** if all required elements (layers, generators, modifiers) are present, even if minor parameter tuning is required. A **Failure** denotes fundamental errors that render the output unusable. We

define a **Mistake** as a suboptimal parameter choice which, while not fatal to the generation process, deviates from the ideal target configuration.

- **Experiment II Metrics:** We track (1) *Token Usage* (summed across Actor and Critic turns), (2) *Follow-up Prompts Required*, and (3) *Objective Completion*. An output is considered complete if the final map matches the target configuration shown in Figure 5 after the allowed iterations and follow up prompts.

4.2 Results I: Reliability and Adherence

Table 2: Comparison of Actor-Critic and Actor+Resources Architectures

Metric	**Actor-Critic**	**Actor+Resources**
Success Rate (%)	80	60
Average mistakes (per successful run)	2.25	2.17
Common Failure Reasons[a]		
Missing Generators/Modifier[b]	1 (50%)	2 (50%)
Missing/wrong parameter choice	0 (0%)	1 (25%)
Did not add reference layer	3 (100%)	2 (50%)
Common Mistakes (Successful Runs)		
Wrong Generator/Modifier Choice[b]	10 (56%)	4 (31%)
Wrong parameter choice	8 (44%)	9 (69%)

[a] A failed run can have more than one failure reason. Percentages represent the fraction of failed runs where each reason was observed.
[b] To generate maps, a Generator/Modifier must first be chosen, determining the PCG algorithm used. Parameters for the chosen Generator/Modifier can then be configured to edit the output.

Quantitative Analysis Evaluated across ten independent trials of the 3D mountain map task, our Actor–Critic architecture achieves an 80% task success rate, representing a 30% relative improvement over the single-agent baseline which achieved 60%. Although the Actor–Critic setup commits slightly more mistakes per successful run than the single-agent setup, these errors are generally non-critical and do not prevent successful map generation. By contrast, the single-agent architecture encounters critical failures more frequently, causing entire runs to fail.

Instruction Following In the six successful trials, the single-agent baseline adhered to the constraint of generating a single mountain in only 50% of cases. As shown in Figure 6, this architecture frequently failed to parameterize the cellular automata base layer correctly, resulting in a network of disjointed land masses rather than a single mass. Conversely, the Actor–Critic architecture complied with this requirement in 88% of trials. Although maps with multiple connected land masses were technically classified as successful runs, these results demonstrate that the dual-agent Actor–Critic setup is substantially more effective at interpreting user instructions and decomposing complex tasks to satisfy multiple prompt constraints.

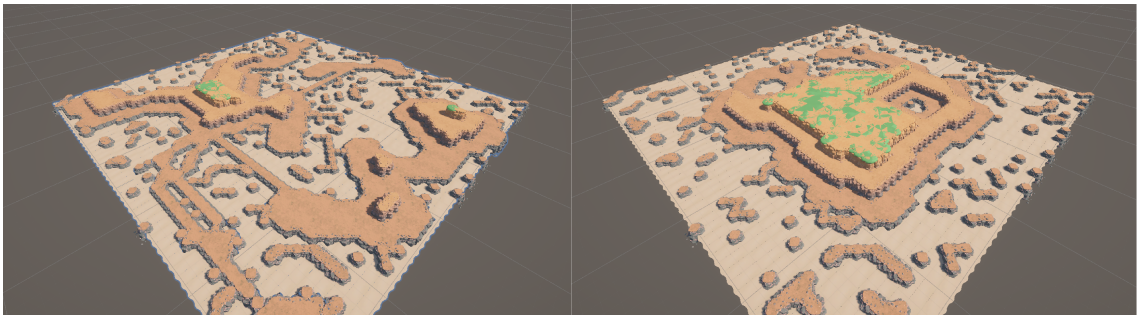


Figure 6: The actor-critic architecture showed more awareness for the requirement of creating one single mountain, configuring the parameters of Cellular Automata to create a single land mass (right) compared to connected masses that the Actor-only architecture created (left).

4.3 Results II: Impact of the Critic Agent

Table 3: Performance comparison of our proposed architecture (Actor-Critic) against baselines.

Map	Model	Tokens used	Prompts required	Achieves objective
2D Beach	Actor-Critic	16,392	2	✓
	Actor+Resources	18,987	4	✓
	Actor[a]	13,884[b]	2[b]	✓[b]
3D Mountain Island	Actor-Critic	14,583	4	✓
	Actor+Resources	11,873	5	✓
	Actor	–	–	×
3D Hilly Golf Course	Actor-Critic	12,633	4	✓
	Actor+Resources	18,676	6	✓
	Actor	–	–	×
2D Escape Maze	Actor-Critic	4,589	2	✓
	Actor+Resources	7,722	3	✓
	Actor	10,774	3	✓

[a] For the Actor column: "–" indicates the method was not applicable or failed to produce a result.
[b] Values in this case were generated by creating C# scripts that adjusted the TileWorldCreator output directly and not through PCG algorithms.

Operational Efficiency and Autonomy A comparison between the Actor–Critic architecture and the single-agent baseline (Actor+Resources) reveals that the primary advantage of the dual-agent system lies in planning quality rather than raw token economy, as shown in Table 3 While the Actor–Critic architecture achieved a modest reduction in total token usage (an average decrease of 12.7% across four tasks), this metric belies a significant shift in workflow. The overhead of the Critic's analysis was offset by a substantial reduction in the need for human intervention. The single-agent baseline required an average of 1.5 additional follow-up prompts per task to correct ineffective strategies or guide the agent toward viable approaches. Consequently, the Actor–Critic setup proved superior not merely in efficiency, but in its ability to autonomously synthesize correct, executable plans without iterative human steering.

Impact of Contextual Resources Beyond the dual-agent dynamic, our results underscore the necessity of domain-specific context. Comparing both architectures against an off-the-shelf LLM baseline (without access to documentation or examples) highlights the significant impact of static resource injection. When supplied with documentation and usage examples, the LLM successfully grounded its generation in the specific functionalities of the *TileWorldCreator* plugin and PCG principles. This domain knowledge proved sufficient for competent content generation even without complex retrieval systems (RAG). These findings suggest that for specialized software engineering tasks, the inclusion of relevant documentation is a foundational requirement that significantly enhances the model's practical utility.

5 Generalizability and Broader Implications

While this work utilizes Procedural Content Generation (PCG) as a testbed, the proposed dual-agent framework addresses a fundamental challenge in software engineering: the *semantic gap* between high-level human intent and the rigid, high-dimensional parameter spaces of expert software.

Bridging the Semantic Gap in Expert Systems. The core innovation of our architecture is not specific to terrain generation; rather, it is a mechanism for *zero-shot interface adaptation*. Traditional approaches to automating complex software rely on task-specific fine-tuning or massive datasets of user interactions. Our results demonstrate that by decoupling *ideation* (Actor) from *verification* (Critic), an LLM can master opaque software parameters solely through static documentation and reference examples. This suggests a scalable pathway for automating interactions with other "expert-grade" software—such as Computer-Aided Design (CAD) tools, scientific simulation platforms (e.g., MATLAB, Ansys), or audio synthesis engines—where parameters are often non-intuitive and the cost of trial-and-error is high.

Democratization of Complex Tools. By effectively translating natural language into executable parameter trajectories, this framework lowers the barrier to entry for professional-grade tools. Users without deep technical knowledge of specific APIs or scripting languages can leverage the full power of the underlying engine. For instance, in Digital Twin rendering or Building Information Modeling (BIM), a user could describe structural requirements in plain English, relying on the Critic to ensure the output adheres to strict engineering constraints defined in the documentation. This shifts the user's role from *operator*—bogged down by syntax and UI navigation—to *director*, focusing purely on high-level design goals while the agents handle the implementation details.

6 Limitations and Future Work

Despite the demonstrated success of the dual-agent framework in zero-shot PCG tasks, several constraints inherent to the current design warrant further investigation. While the system effectively bridges the semantic gap for single-session interactions, broader deployment would require addressing computational overhead and the lack of cross-session learning. The following subsections outline these limitations and propose avenues for future development.

Inference Latency and Computational Cost. Although the Actor–Critic architecture significantly improves generation quality, the iterative nature of the feedback loop inevitably incurs higher latency and token costs compared to single-shot baselines. The Critic's verification step doubles the inference requirement per iteration. Future work will investigate optimization strategies to mitigate this overhead, such as caching successful parameter trajectories for reuse or employing smaller, distilled models (e.g., Llama-3-8B or Phi-3) specifically fine-tuned for the Critic's verification role, thereby reducing cost without compromising reliability.

Systematic Bias Analysis. Our experiments revealed that both architectures occasionally exhibited consistent error patterns, such as specific spatial misalignments or recurrent hallucinations of non-existent API parameters. Currently, our system corrects these biases reactively via the Critic. A key area for future work is the development of a *diagnostic meta-layer* that systematically categorizes recurring failures. This would enable the transition from reactive error correction to proactive error prevention, potentially by dynamically adjusting the system prompt based on detected error trends.

Long-term Memory and Adaptation. The current system operates episodically, resetting its context buffer between tasks. Consequently, the agents cannot "learn" from previous successes or failures across different sessions. A promising direction for future research is the integration of Retrieval-Augmented Generation (RAG). By archiving successful strategies and resolved failure modes into a vector database, the system could retrieve relevant "experiences" for new, unseen tasks. This would allow the agents to adapt over time and improve zero-shot performance on novel maps without the need for resource-intensive parameter updates.

7 Conclusions

We have presented a dual-agent architecture that enables Large Language Models (LLMs) to master complex 3D procedural content generation (PCG) tasks in Unity. By decoupling generation from verification, our Actor-Critic framework achieves high-quality, zero-shot map creation, effectively bridging the semantic gap between natural language and opaque domain-specific parameters. Crucially, our results demonstrate that resource-intensive fine-tuning is not required for domain mastery; instead, providing off-the-shelf models with documentation and an iterative feedback loop is sufficient for robust tool operation. This approach significantly outperforms single-agent baselines in interpretable task decomposition and constraint satisfaction. Ultimately, this work offers a scalable and training-free paradigm for generative design, providing a flexible blueprint that can be readily transferred to other software tools and APIs.

References

[1] Sajad Mohaghegh, Mohammad Amin Ramezan Dehnavi, Golnoosh Abdollahinejad, and Matin Hashemi. Pcgpt: Procedural content generation via transformers, 2023.

[2] Graham Todd, Sam Earle, Muhammad Umair Nasir, Michael Cerny Green, and Julian Togelius. Level generation through large language models. In *Proceedings of the 18th International Conference on the Foundations of Digital Games*, FDG 2023, page 1–8. ACM, April 2023.

[3] J. Togelius, A.J. Champandard, Pier Luca Lanzi, M. Mateas, A. Paiva, Mike Preuss, and K.O. Stanley. Procedural content generation: goals, challenges and actionable steps. *Dagstuhl Follow-Ups*, 6:61–75, 01 2013.

[4] Yuzhong Zhang, Guixuan Zhang, and Xinyuan Huang. A survey of procedural content generation for games. In *2022 International Conference on Culture-Oriented Science and Technology (CoST)*, pages 186–190, 2022.

[5] Mojang Studios. Minecraft. Video game, 2011. Multiplatform.

[6] Blizzard North. Diablo. Video game, 1997. Blizzard Entertainment, Ubisoft and Electronic Arts.

[7] Iron Gate Studio. Valheim. Video game, 2021. Early access; Microsoft Windows; published by Coffee Stain Studios.

[8] Noor Shaker, Julian Togelius, and Mark J Nelson. *Procedural content generation in games*. Computational Synthesis and Creative Systems. Springer International Publishing, Cham, Switzerland, 1 edition, October 2016.

[9] Julian Togelius, Georgios N. Yannakakis, Kenneth O. Stanley, and Cameron Browne. Search-based procedural content generation: A taxonomy and survey. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(3):172–186, 2011.

[10] Oliver Withington and Laurissa Tokarchuk. The right variety: Improving expressive range analysis with metric selection methods. In *Proceedings of the 18th International Conference on the Foundations of Digital Games*, FDG 2023, page 1–11. ACM, April 2023.

[11] Yuqian Sun, Zhouyi Li, Ke Fang, Chang Hee Lee, and Ali Asadipour. Language as reality: A co-creative storytelling game experience in 1001 nights using generative ai, 2023.

[12] Andrew Zhu, Lara Martin, Andrew Head, and Chris Callison-Burch. Calypso: Llms as dungeon master's assistants. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 19(1):380–390, October 2023.

[13] Shyam Sudhakaran, Miguel González-Duque, Claire Glanois, Matthias Freiberger, Elias Najarro, and Sebastian Risi. Mariogpt: Open-ended text2level generation through large language models, 2023.

[14] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.

[15] Mengchao Ren. Advancements and applications of large language models in natural language processing: A comprehensive review. *Applied and Computational Engineering*, 97(1):55–63, November 2024.

[16] Muhammad U Nasir and Julian Togelius. Practical pcg through large language models, 2023.

[17] In-Chang Baek, Tae-Hwa Park, Jin-Ha Noh, Cheong-Mok Bae, and Kyung-Joong Kim. Chatpcg: Large language model-driven reward design for procedural content generation, 2024.

[18] Sihao Hu, Tiansheng Huang, Gaowen Liu, Ramana Rao Kompella, Fatih Ilhan, Selim Furkan Tekin, Yichang Xu, Zachary Yahn, and Ling Liu. A survey on large language model-based game agents, 2025.

[19] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.

[20] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022.

[21] Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don't stop pretraining: Adapt language models to domains and tasks, 2020.

[22] Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time, 2022.

[23] Mahdi Farrokhi Maleki and Richard Zhao. Procedural content generation in games: A survey with insights on emerging llm integration. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 20(1):167–178, November 2024.

[24] In-Chang Baek, Tae-Gwan Ha, Tae-Hwa Park, and Kyung-Joong Kim. Toward cooperative level generation in multiplayer games: A user study in overcooked! In *2022 IEEE Conference on Games (CoG)*, pages 276–283, 2022.

[25] Marcus Gerhold and Kristian Tijben. Computer aided content generation – a gloomhaven case study. In *Proceedings of the 18th International Conference on the Foundations of Digital Games*, FDG '23, New York, NY, USA, 2023. Association for Computing Machinery.

[26] Adam Summerville, Sam Snodgrass, Matthew Guzdial, Christoffer Holmgård, Amy K. Hoover, Aaron Isaksen, Andy Nealen, and Julian Togelius. Procedural content generation via machine learning (pcgml), 2018.

[27] Marton Szep, Daniel Rueckert, Rüdiger von Eisenhart-Rothe, and Florian Hinterwimmer. Fine-tuning large language models with limited data: A survey and practical guide, 2025.

[28] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020. The seminal paper establishing In-Context Learning (ICL).

[29] Archit Sharma, Sedrick Keh, Eric Mitchell, Chelsea Finn, Kushal Arora, and Thomas Kollar. A critical evaluation of ai feedback for aligning large language models, 2024.

[30] André Linard Santos Auxtero. Game environment design creator using artificial intelligence procedural generation. Master's thesis, Universidade NOVA de Lisboa (Portugal), 2023.

[31] Yi-Chun Chen and Arnav Jhala. Narrative-to-scene generation: An llm-driven pipeline for 2d game environments, 2025.

[32] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023.

[33] Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, et al. Augmented language models: a survey. *arXiv preprint arXiv:2302.07842*, 2023.

[34] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[35] Song Tang, Kaiyong Zhao, Lei Wang, Yuliang Li, Xuebo Liu, Junyi Zou, Qiang Wang, and Xiaowen Chu. UnrealLLM: Towards highly controllable and interactable 3D scene generation by LLM-powered procedural content generation. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Findings of the Association for Computational Linguistics: ACL 2025*, pages 19417–19435, Vienna, Austria, July 2025. Association for Computational Linguistics.

[36] Ziniu Hu, Ahmet Iscen, Aashi Jain, Thomas Kipf, Yisong Yue, David A. Ross, Cordelia Schmid, and Alireza Fathi. Scenecraft: An llm agent for synthesizing 3d scene as blender code, 2024.

[37] Sining Lu, Guan Chen, Nam Anh Dinh, Itai Lang, Ari Holtzman, and Rana Hanocka. Ll3m: Large language 3d modelers, 2025.

[38] Chunyi Sun, Junlin Han, Weijian Deng, Xinlong Wang, Zishan Qin, and Stephen Gould. 3d-gpt: Procedural 3d modeling with large language models, 2024.